



Universidad Nacional Autónoma de México

Facultad de Ingeniería

Modelos de Programación Orientada a Objetos

Profesor(a):

Prof. Oscar Manuel Ruiz Hurtado.

Semestre:

2027-1

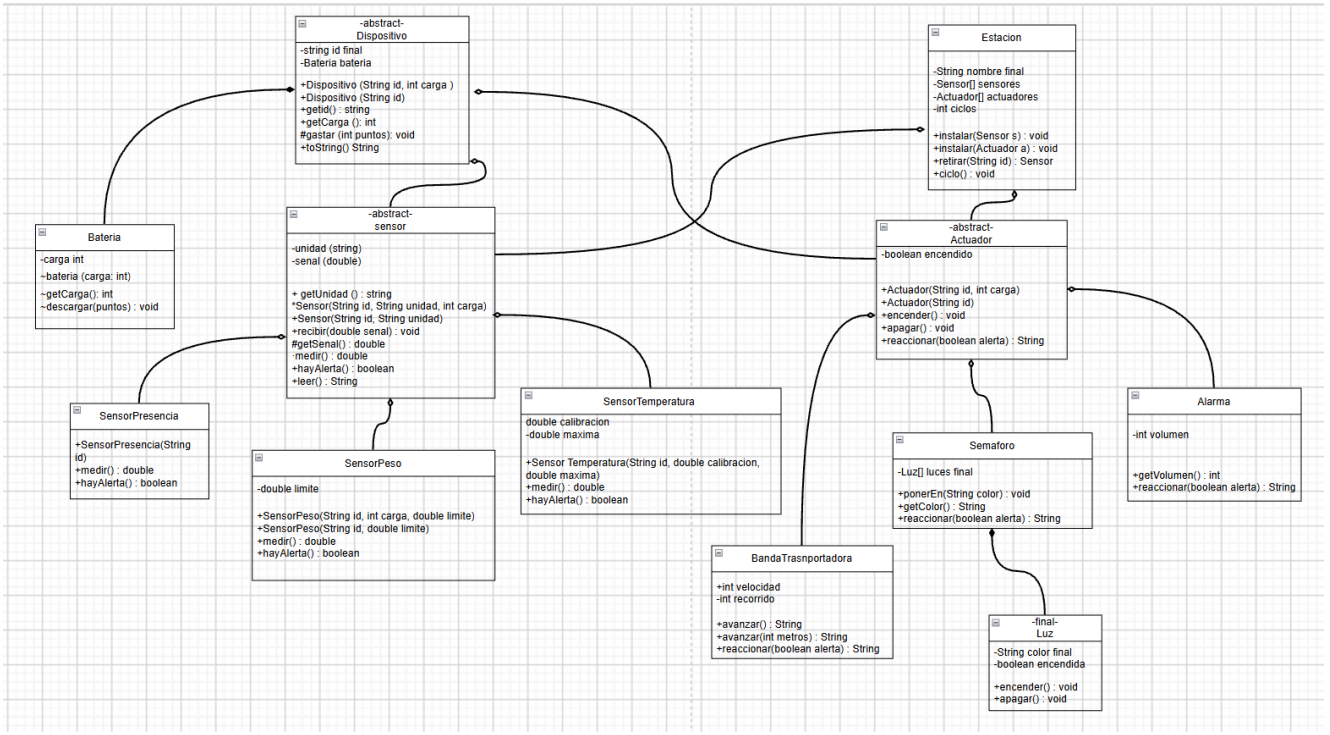
Practica: 8

Grupo: 6

Integrantes:

Crisostomo Garcia Israel

1. Diagrama de Clases



2. Ejecución del Programa

```
israc@ISRAELCG MINGW64 ~/ISRAELCRIS/mpoo-2027-1/entregas/crisostomo_israel/p08 (entregas_crisostomo_israel)
$ java -cp out estacion.PruebaTemperatura
Sensor S3 (100%) temperatura, max 45.0
S3 mide 40.0 grados
S3 mide 48.5 grados ALERTA
Sensor S3 (98%) temperatura, max 45.0

israc@ISRAELCG MINGW64 ~/ISRAELCRIS/mpoo-2027-1/entregas/crisostomo_israel/p08 (entregas_crisostomo_israel)
$ java -cp out estacion.Main
Instalado: Sensor S1 (100%) presencia
Instalado: Sensor S2 (80%) peso, limite 25.0
Instalado: Sensor S3 (100%) temperatura, max 45.0
Instalado: Actuador A1 (100%) banda, 2 m por ciclo
Instalado: Actuador A2 (100%) semaforo en apagado
Instalado: Actuador A3 (100%) alarma, volumen 10
== Ciclo 1 en Anden 3
  S1 mide 12.0 cm
  S2 mide 18.5 kg
  S3 mide 40.0 grados
  A1 avanza 2 m, lleva 2 m
  A2 en verde
  A3 en silencio
== Ciclo 2 en Anden 3
  S1 mide 9.5 cm
  S2 mide 32.0 kg ALERTA
  S3 mide 42.0 grados
  A1 se detiene
  A2 en rojo
  A3 suena a volumen 10
Retirado: S2
== Ciclo 3 en Anden 3
  S1 mide 15.0 cm
  S3 mide 46.5 grados ALERTA
  A1 se detiene
  A2 en rojo
  A3 suena a volumen 10
Regresa: Sensor S2 (78%) peso, limite 25.0
Instalado: Sensor S2 (78%) peso, limite 25.0
A1 avanza 5 m, lleva 7 m
Anden 3: 3 sensores, 3 actuadores, 3 ciclos

israc@ISRAELCG MINGW64 ~/ISRAELCRIS/mpoo-2027-1/entregas/crisostomo_israel/p08 (entregas_crisostomo_israel)
$ git status
On branch entregas_crisostomo_israel
Your branch is ahead of 'origin/entregas_crisostomo_israel' by 10 commits.
  (use "git push" to publish your local commits)
```

3. Respuestas Teóricas

1. Sobrecarga contra sobrescritura: La **sobrecarga** ocurre cuando existen métodos con el mismo nombre en la misma clase pero con distintos parámetros. Un ejemplo en nuestro código está en BandaTransportadora, que tiene avanzar(int metros) y avanzar() (sin parámetros). La **sobrescritura** se da cuando una clase hija redefine un método heredado de su clase padre. Un ejemplo es el método abstracto medir(), el cual sobrescribimos usando @Override en SensorTemperatura y SensorPeso para darle su propio comportamiento.

2. Composición contra agregación: La **composición** implica una dependencia fuerte (si el dueño muere, la parte también). La Bateria es composición porque Dispositivo la crea internamente con new Bateria(); no puede existir por sí sola. La **agregación** es una relación más débil donde la parte sobrevive si la sacan del contenedor. Los sensores son agregación para la Estacion porque se crean fuera (en el Main). Esto se nota en el ciclo 3: cuando la estación usa retirar("S2"), la báscula no se destruye, sino que sigue viva en memoria y más adelante regresa siendo el mismo objeto con su batería intacta al 78%.

3. Modificadores: Al agregar bascula.gastar(5); en Main, el compilador arroja este error:error: gastar(int) has protected access in dispositivos.Dispositivo

Por qué ocurre: El método gastar está definido como protected en la clase base Dispositivo. Este modificador restringe su uso exclusivamente a las clases dentro de su mismo paquete (dispositivos) o a sus clases hijas (como SensorPeso). Como Main pertenece a un paquete completamente distinto (estacion) y no hereda de Dispositivo, Java bloquea la llamada por protección de acceso.